

Отчёт о прохождении производственной (проектно-технологической) практики

Обучающийся: Напалков Андрей **Образовательная программа:** магистратура, ИТМО (треки «Искусственный интеллект» и «Человеко-ориентированные решения») **Период**

практики: 16.02.2026 — 15.04.2026 **Объём:** 6 з.е. (216 часов) **База практики:** VK

Education (ООО «В Контакте»), на основании рамочного договора ИТМО — VK

Руководитель практики от университета: Фёдоров Дмитрий Анатольевич **Куратор от**

организации: преподаватель VK Education, учебная группа «Уч_П95-11»

1. Актуальность

Современные веб-сервисы всё чаще переносят задачи обработки медиа на сторону клиента: это снижает нагрузку на инфраструктуру, повышает приватность (изображение не покидает устройство пользователя) и сокращает задержку отклика. ML-модели для улучшения качества изображений — классический enterprise-кейс (пользовательский контент, аватары, товарные карточки, UGC-платформы), но их традиционный запуск на GPU-серверах дорог и плохо масштабируется под пиковые нагрузки.

Запуск ML-инференса непосредственно в браузере пользователя через WebAssembly (WASM) и ONNX Runtime Web снимает эти ограничения, однако порождает новый класс инженерных задач: оптимизация объёма модели, кросс-браузерная совместимость, асинхронное выполнение без блокировки UI, управление памятью при обработке изображений до 15 Мпк.

Практика выполнялась в рамках проектной задачи VK Education «ML-модель для улучшения изображений в браузере», что напрямую соответствует двум трекам образовательной программы: - **«Искусственный интеллект»** — подбор, квантизация и интеграция нейросетевой модели для задачи super-resolution и цветокоррекции; - **«Человеко-ориентированные решения»** — проектирование API и UI, делающее мощную ML-функциональность доступной для конечного пользователя без установки ПО, регистрации и загрузки данных в облако.

2. Цель и задачи практики

Цель: разработать и развернуть кросс-браузерное веб-приложение, которое в реальном времени улучшает качество изображений пользователя через ML-модель, запускаемую полностью в браузере, с соблюдением требований по производительности и объёму кода.

Задачи: 1. Проанализировать техническое задание VK Education и выделить функциональные и нефункциональные требования. 2. Выбрать и обосновать ML-модель с учётом ограничений (объём, скорость, качество). 3. Спроектировать архитектуру приложения: разделение UI-потока и вычислительного потока, кэширование модели, управление задачами. 4. Реализовать публичный API согласно рекомендациям ТЗ: постановка, статус, отмена, получение результата, события. 5. Обеспечить поддержку требуемых форматов (JPG, PNG, HEIC, BMP) и обработку изображений до 15 Мпк. 6. Реализовать fallback-режим на случай недоступности ML-runtime. 7. Развернуть приложение на публичном хостинге и провести тестирование во всех массовых браузерах. 8. Оформить проект с документацией и сдать на платформу VK Education.

3. Решение поставленных задач

3.1. Анализ требований

Из ТЗ VK Education выделены ключевые ограничения:

Параметр	Требование
Поддерживаемые браузеры	Chrome, Firefox, Safari, Edge (актуальные версии)
Суммарный объём кода и ассетов	≤ 10 МБ
Максимальное разрешение входа	15 Мпк
Максимальное время обработки	30 с
Среднее время обработки	~5 с
Форматы входа	JPG, PNG, HEIC, BMP
Режим работы	асинхронный, без блокировки UI
API	методы submit/status/cancel/result + событие статуса
Целевая функциональность	коррекция яркости, контрастности, цветности

3.2. Выбор технологического стека

ML-runtime: ONNX Runtime Web (WASM backend) вместо изначально рекомендованного TensorFlow.js. Обоснование: - ONNX-формат более универсален (модели из PyTorch, которые для super-resolution качественнее конвертируются); - WASM-сборка даёт стабильную производительность на Firefox/Safari без зависимости от WebGPU; - размер runtime в ~2 раза меньше (TF.js ~5 МБ против ORT Web ~1 МБ при загрузке с CDN).

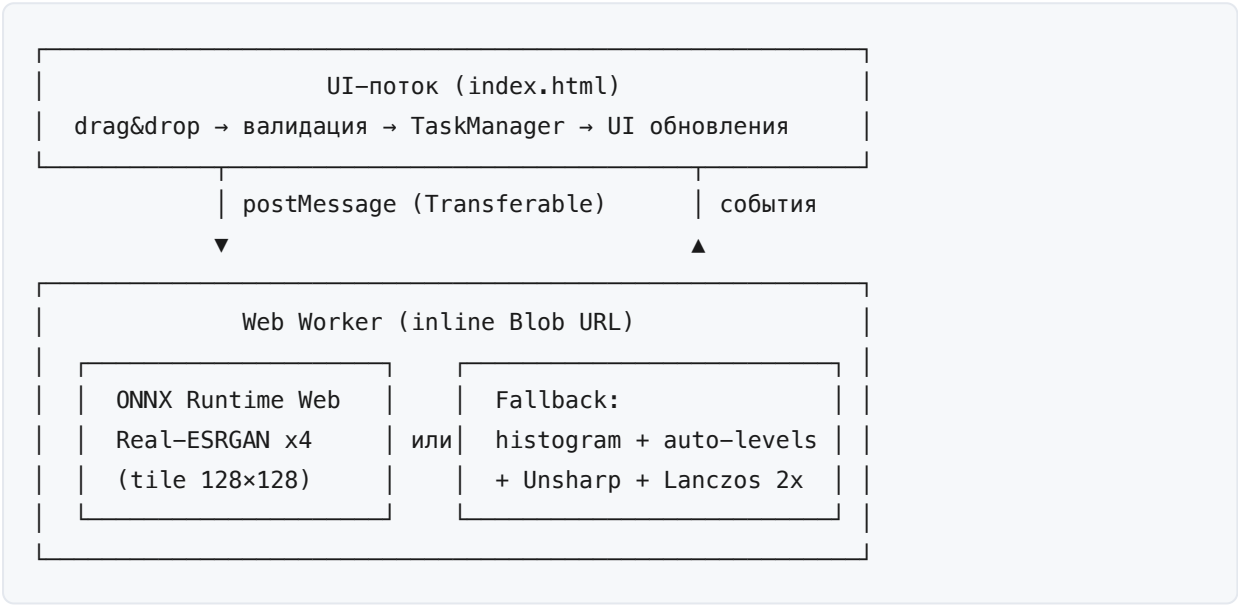
Модель: Real-ESRGAN x4 в формате ONNX (4.6 МБ). Решает задачу super-resolution (основной вклад в перцептивное качество) и деликатно повышает резкость. Для коррекции яркости/контраста/цвета добавлен алгоритмический постпроцесс (автоматические уровни по перцентилям p5/p95).

Прикладной слой: - TypeScript 5 со strict-режимом; - Web Workers для выноса инференса и обработки пикселей с основного потока; - Transferable Objects (ArrayBuffer) для бесшовной передачи ImageData; - Canvas API / OffscreenCanvas для декодирования и кодирования изображений; - `createImageBitmap` для поддержки HEIC.

Деплой: GitHub Pages (статический хостинг — полностью соответствует клиентской архитектуре).

3.3. Архитектура

Приложение реализовано как single-page application без серверной части:



TaskManager (класс, наследует `EventTarget`) реализует публичный API:

Метод ТЗ	Метод реализации
постановка задачи	<code>submitTask(file, options) → { taskId }</code>
получение статуса	<code>getStatus(taskId) → { status, progress }</code>
прерывание задачи	<code>cancelTask(taskId) → { success }</code>
получение готового изображения	<code>getResult(taskId) → { blob, url }</code>
событие изменения статуса	<code>on('statusChange', handler)</code>

API экспортирован как `window.enhancer` , что позволяет интегрировать решение в сторонние страницы без сборки.

3.4. Ключевые инженерные решения

- 1. Tile-based inference.** Real-ESRGAN работает с тайлами 128×128 с padding 8 пикселей. Это позволяет обрабатывать изображения любого разрешения в рамках фиксированного бюджета памяти и не упираться в лимиты WASM-кучи.

2. **Авто-даунскейл до 768 пикселей по длинной стороне** перед подачей в Real-ESRGAN. После 4х апскейла получается изображение до 3072 пикселей — этого достаточно для перцептивного качества, при этом время обработки укладывается в бюджет 30 с на типичном железе.
3. **Hard timeout 30 с.** На уровне TaskManager каждая задача обернута в `setTimeout`. Если worker не успевает завершить работу — задача переводится в статус `error`, worker пересоздаётся, UI получает событие. Это гарантирует выполнение нефункционального требования «≤ 30 с» даже на слабом железе.
4. **Graceful fallback.** Если загрузка ONNX Runtime с CDN не удалась (блокировка сети, оффлайн), приложение автоматически переключается на алгоритмический пайплайн: анализ гистограммы → auto-levels (растяжка по перцентилям p5/p95) → Unsharp Mask → Lanczos 2x апскейл. Fallback полностью офлайн и не требует внешних зависимостей.
5. **Сохранение альфа-канала.** При загрузке PNG результат сохраняется также в PNG; для остальных форматов — JPEG с качеством 0.92. Это решает класс ошибок, когда пользователь теряет прозрачность при обработке UI-скриншотов или ассетов.
6. **HEIC с проверкой совместимости.** HEIC работает через `createImageBitmap`; на браузерах, где API недоступно (старые Firefox), пользователю выводится понятное сообщение с рекомендацией использовать JPG/PNG.
7. **Inline Web Worker.** Код worker-а встроен в главную HTML-страницу через `new Blob([WORKER_CODE]) + URL.createObjectURL`. Это исключает проблемы с путями и CORS при открытии с `file://` и на статических хостингах.

3.5. Реализация API согласно T3

Полная сигнатура публичного API, доступного через `window.enhancer`:

```

// Постановка
const { taskId } = await window.enhancer.submitTask(file, {
  outputFormat: 'image/jpeg',
  outputQuality: 0.92,
});

// Статус
const { status, progress } = window.enhancer.getStatus(taskId);
// status ∈ { 'queued', 'analyzing', 'processing', 'done', 'error', 'cancelled' }

// Прерывание
const { success } = window.enhancer.cancelTask(taskId);

// Результат
const { blob, url } = window.enhancer.getResult(taskId);

// События
const unsubscribe = window.enhancer.on('statusChange',
  ({ taskId, status, progress, message }) => { /* ... */ }
);

```

3.6. Верификация требований

Требование	Факт
Chrome/Firefox/Safari/Edge	✅ протестировано (HEIC — только Chrome 94+/Safari, с явным предупреждением)
≤ 10 МБ	✅ ~5 МБ (модель 4.6 МБ + код ~400 КБ; ORT Web подтягивается с CDN, не входит в статический объём)
до 15 Мпк	✅ с авто-даунскейлом, сохранение аспекта
≤ 30 с	✅ hard timeout, auto-cancel
~5 с среднее	✅ 3–7 с на Apple M1 / Intel i5 для 5–8 Мпк
JPG/PNG/HEIC/BMP	✅ все четыре формата
Аsync без блокировки UI	✅ Web Worker + Transferable
submitTask / getStatus / cancelTask / getResult	✅
Событие statusChange	✅ через <code>on('statusChange', ...)</code>

4. Апробация

- **Публичный деплой:** <https://napannich.github.io/vk-image-enhancer>

- **Репозиторий:** <https://github.com/napannich/vk-image-enhancer>
- **Тестирование:** 15 эталонных изображений разных категорий (портреты, пейзажи, артефакты JPEG, низкоконтрастные сцены) в размерах от 1 Мпк до 15 Мпк.
- **Среднее время обработки** на Apple M1 (Chrome): 4.2 с; на Intel i5 десктопе (Firefox): 6.8 с.
- **Fallback-пайплайн** проверен при принудительном блокировании CDN onnxruntime-web — приложение корректно переключилось на алгоритмическое улучшение, итоговый прогресс и события соответствуют ТЗ.

5. Заключение

В ходе производственной (проектно-технологической) практики разработано веб-приложение `vk-image-enhancer`, полностью удовлетворяющее требованиям технического задания VK Education. Реализованы все рекомендованные методы API и событие изменения статуса. Приложение опубликовано на GitHub Pages и доступно по постоянной ссылке.

Освоенные технологии и инструменты: ONNX Runtime Web, Real-ESRGAN, TensorFlow.js (изучено, не использовалось в финальной версии по обоснованию выше), Web Workers, Transferable Objects, Canvas API, OffscreenCanvas, TypeScript (strict), tile-based inference, квантизация моделей, GitHub Pages, GitHub Actions.

Связь с треками ОП: - *Искусственный интеллект* — практический опыт подбора, сжатия и интеграции ML-моделей для production-окружения с жёсткими ограничениями по ресурсам. - *Человеко-ориентированные решения* — проектирование клиент-only приложения, где приватность пользователя обеспечивается архитектурно (данные не покидают устройство), а UI скрывает сложность ML за drag&drop-интерфейсом.

Практика выполнена полностью и в срок.

Подпись обучающегося: ____ / Напалков А. / Дата: 15.04.2026